

UC MATH 600 MATHEMATICS FOR COMPUTER SCIENCE · MODULE 01 · LECTURE NOTES

What a Proof Is For

A tool for being sure, not a ritual for being graded.

MODULE OUTCOMES

- State what a proof establishes and what it does not.
- Distinguish a proof from an argument and from evidence.
- Explain the role of counterexamples and testing.
- Read a proof actively rather than passively.
- State why this course is a prerequisite.

1 What a proof is

A PROOF IS A CHAIN OF STEPS EACH OF WHICH A READER CAN CHECK, ENDING WHERE YOU WANTED TO BE

Not a demonstration that you are clever, and not a ritual that a grader requires — a device for transferring certainty from premises to a conclusion, one checkable step at a time.

And the reader it is written for is somebody who does not already believe you — which is what determines the level of detail, and is the only question to ask when deciding what to include: would a sceptical reader accept this line without further argument?

So the word 'obviously' is a warning sign — it marks exactly the place where you stopped checking — and in this program's experience it marks the place where the error is. Replace it with the reason, or delete the step.

Which is why proofs are written out at all: you are the reader most likely to be fooled by your own argument, since you already believe the conclusion — **and writing it out in full is the cheapest available defence against that** (Module 13 §2).

2 Proof, argument, evidence

Kind	What it gives you	What it cannot do
Proof	Certainty, conditional on the premises.	Tell you the premises are true — which is a separate question and frequently the harder one.

Kind	What it gives you	What it cannot do
Argument	Reason to believe, and often the right intuition.	Rule out the case you did not think of, which is what the formal version is for.
Evidence (testing)	Confirmation on the cases you tried.	Cover the infinitely many you did not try.
Counterexample	Certainty that the statement is false.	— nothing; it is complete on its own.

A single counterexample is complete and a million confirming cases are not — which is the asymmetry that makes testing cheap and proving necessary, and which is the whole justification for §2's callout. The asymmetry between confirming and refuting is this module's practical content, and it reappears in CSCE 629 as the difference between a correctness proof and a passing test suite.

WHICH IS WHY TESTING BEFORE PROVING IS GOOD PRACTICE RATHER THAN CHEATING

Check your conjecture on ten thousand cases first — four lines of Python — and if it fails, you have saved yourself a day and gained a counterexample, which is itself a complete result (§2's table).

And if it holds, you have learned nothing about whether it is true — no amount of testing establishes a universal statement — but you now know it is worth the effort of proving, which is a real saving over discovering at the end of a long argument that the statement was false.

Plus the failures are informative: the smallest counterexample usually shows you precisely which hypothesis you forgot — and that frequently converts a false conjecture into a true one with an extra condition attached, which is how most theorems actually get their hypotheses.

So the workflow is: conjecture, test, then prove — and Project 1 requires one of each, including one conjecture you disproved rather than proved, because finding a counterexample is a skill with its own technique.

3 Reading proofs

- **Read the statement first and try it yourself for ten minutes — which is the only way the proof's moves become visible as choices** rather than as inevitable steps, and ten minutes of failing is worth an hour of reading.
- **Then read for the *idea*, not the steps — most proofs have one idea and a page of verification, and the idea is usually expressible in a single sentence.** If you can state that sentence you have the proof; the rest you can reconstruct.
- **Then go back and check every step**, including and especially the ones that look routine — **which is where published errors actually live**, since nobody checks them.
- **Then ask where each hypothesis was used — a hypothesis that is never used is either unnecessary (so the theorem is stronger than stated) or the proof is wrong — and finding out which teaches more than the proof did.**
- **And then close the book and rewrite it from memory**, which is **the step that distinguishes having read a proof from being able to produce one**, and is the only part of this list that is uncomfortable.

4 Why this is a prerequisite

CSCE 629 IS WRITTEN IN THIS LANGUAGE AND WILL NOT STOP TO TRANSLATE

'We proceed by induction on the number of edges'; 'the recurrence solves to $O(n \log n)$ by the master theorem'; 'with probability at least $1 - 1/n$ the algorithm succeeds' — three sentences from the first few weeks of Semester 1, none of which that course will stop to explain, because it assumes this one.

And the failure mode is not confusion but slow attrition: you follow eighty per cent of each lecture, fall a little further behind every week, and eventually conclude that the subject is beyond you — which it is not; the notation was.

So the honest test is Module 13's self-check, taken before Semester 1 rather than after — if you can answer it, skip this course with a clear conscience — and four to six weeks here is the stated cost of being able to.

Which is a short time measured against a two-year program, and it is the highest-return four weeks in the whole thing for anybody who needs it. It is also the easiest course here to convince yourself you do not need.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Lehman, Leighton & Meyer, chapter 1 (free)	§§1 and 2 — what a proof is, with the false proofs that make the point better than true ones do.
Velleman — How to Prove It, chapter 1	§1 — proof as a transferable skill, patiently. Library copy.
Pólya — How to Solve It	§3 — how to approach a problem you cannot yet do, which is the actual difficulty. Library copy.
Project Euler (free)	§2's workflow — problems where testing a conjecture computationally is the natural first move.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write out a proof you already believe** in full, and mark every step you had to think about.
- 2 **Find three uses of 'obviously'** in any textbook and check whether each is.
- 3 **State the difference** between a proof and evidence, in two sentences.
- 4 **Find a statement true for n up to 40 and false after**, and say what that shows.
- 5 **Write a four-line test** for a conjecture before proving it.
- 6 **Disprove something by counterexample**, found by search.
- 7 **Then add a hypothesis** that makes it true.
- 8 **Read one proof and state its single idea** in a sentence.
- 9 **Find a hypothesis in a proof** and locate exactly where it is used.
- 10 **Take Module 13's self-check now**, and record your score honestly.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is a proof for, and who is it written for?
- 2 Why is 'obviously' a warning sign?
- 3 Why are you the reader most likely to be fooled?
- 4 Contrast proof, argument, evidence, and counterexample.
- 5 State the asymmetry between confirming and refuting.
- 6 Why is testing before proving good practice?

- 7 What does a failed test give you beyond a saved day?
- 8 Give the five steps of reading a proof actively.
- 9 What does an unused hypothesis tell you?
- 10 What is the prerequisite failure mode, and why is it hard to notice?